

Lab0: DataLab

熊思宇 24307130130

1. P1

```
148 int signMask(void) {
149     return 1 << 31;
150 }
```

需要返回最高位为 1 的掩码 0x80000000。将整数 1 左移 31 位 ($1 \ll 31$)，使 1 移到最高位即可

2. P2

```
160 int bitXor(int x, int y) {
161     return ~(~x & ~y) & ~(x & y);
162 }
```

异或即两个中位恰好一个为 1。 $\sim(\sim x \& \sim y)$ 表示「至少一个为 1」， $\sim(x \& y)$ 表示并非两个都为 1，两者相与即为异或

3. P3

```
172 int negativePart(int x){
173     int mask = x >> 31; // 符号掩码：负数为1，非负为0
174     return (~x + 1) & mask; // -x 与掩码&，负数取-x，非负取0
175 }
```

利用 $x \gg 31$ 得到符号掩码 ($x < 0$ 时 mask 为 1，否则为 0)，再用 $-x = \sim x + 1$ 得到 $-x$ ，将 $(\sim x + 1)$ 与掩码 $\&$ ，即可在 $x < 0$ 时返回 $-x$ 、否则返回 0

4. P4

```
188 int copyByteWithin(int x, int src, int dst) {
189     int byte = (x >> (src << 3)) & 0xFF; // 提取src位置字符
190     int mask = ~(0xFF << (dst << 3)); // 掩码记录待删除的位置
191     return (x & mask) | (byte << (dst << 3));
192 }
```

将 x 右移 $\text{src} \times 8$ 位后与 0xFF 取 $\&$ ，提取出第 src 处的字节，再用 $\sim(0xFF \ll \text{dst} \times 8)$ 构造 dst 处的清除掩码，用于将 x 的第 dst 字节清零，再把提取出的字节左移 $\text{dst} \times 8$ 位到目标位置，再相或得到结果。

5. P5

```
203 int logicalShift(int x, int n) {
204     int a = x >> n; // 算术右移（但会补符号位1）
205     int b = (1 << 31) >> n;
206     int mask = ~(b << 1); // 不使用-，先右移n再左移1，然后取反
207     return a & mask;
```

此题的逻辑右移需在高位补 0，而 C 对有符号数的 $\gg$ 是算术右移，因此需要

先做算术右移，再用掩码将前 n 位清零，基于 P1，再右移取反得到前 n 位为 0 的掩码，再取&即可

6. P6

```
218 int swapNibblePairs(int x) {  
219     int m = 0x0F | (0x0F << 8); // 0x0F0F  
220     m = m | (m << 16); // 0x0F0F0F0F  
221     int low = (x & m) << 4; // 低半字节移到高位  
222     int high = (x >> 4) & m; // 高半字节移到低位  
223     return low | high; // 合并
```

先拼出掩码 0x0F0F0F0F（每字节低 4 位）。低半字节通过 $(x \& m) \ll 4$ 移到高位，高半字节通过 $(x \gg 4) \& m$ 移到低位最后相或合并

7. P7

```
235 ~ int secondLowestZeroBit(int x) {  
236     int low0 = ~x & (x + 1); // 找最低的0位  
237     int x2 = x | low0; // 把最低0位变为1  
238     return ~x2 & (x2 + 1); // 再找现在的最低0位，即原本的第二低0位
```

$\sim x \& (x + 1)$: $x + 1$ 的进位在最低 0 位，0 变为 1，取 & 时，由于高位 $\sim x$ 与 $x + 1$ 都不同，最后全部变为 0，低位都是 0，因此 $\sim x \& (x + 1)$ 提取的是最低 0 位。重复两次得第二低 0 位

8. P8

```
250 int oddParity(int x) {  
251     x = x ^ (x >> 16);  
252     x = x ^ (x >> 8);  
253     x = x ^ (x >> 4);  
254     x = x ^ (x >> 2);  
255     x = x ^ (x >> 1);  
256     return !(x & 1);
```

异或时，只有 1 和 0 一起时，结果为 1, 00 和 11，结果都为 0，因此可以用异或表示 1 的个数，不断对半分进行异或，将 32 位逐步两两异或，最终把所有位的异或结果集中到最低位，异或为 0 表示 1 的个数为偶数，为 1 表示奇数，再对结果取 ! 即可。

9. P9

```
268 int rotateRightBits(int x, int n) {
269     int m = n & 31;           // 取模
270     int y = (~m + 1) & 31;    // 左移对应的位数
271     int left = x << y;
272     int mask = ~(~0 << y);    // 顶部m位为0的掩码，用于处理right
273     int right = (x >> m) & mask; // 与mask作&，去掉高位可能补的1
274
275     return left | right;
276 }
```

本题拆为左移和右移两部分，低 n 位左移到顶部、其余位右移 n 位， n 未限制范围，因此要先 $n \& 31$ 对 n 取模，再分别处理左移和右移部分，最后相或即可。

10. P10

```
289 int roundEvenPow2(int x, int n) {
290     int m = 1 << n;           //  $2^n$ 
291     int t = (x >> n) & 1;     // 奇偶判据（偶数商=0，奇数商=1）
292     int bias = (m >> 1) + ~0; //  $m/2 - 1$ 
293     return ((x + bias + t) >> n) << n; // 加偏移后四舍五入
294 }
```

取 $m=2^n$ ， $(X \gg n) \ll n$ 可以舍弃 n 后面的所有位数，且舍弃后恰好是 m 的整数倍，达到需求只需要再加上 $(m/2 - 1)$ 的偏置项，其中 -1 是为了结合 t 进行修正， $t = (x \gg n) \& 1$ 取最低位的奇偶性，确保向商为偶数的倍数舍入，当余数小于 $m/2$ 时不进位、大于时进位，恰好居中时，根据奇偶位决定。

11. P11

```
308 int midpointTowardFirst(int x, int y) {
309     int t = (x & y) + ((x ^ y) >> 1); // 向下取整
310     int odd = (x ^ y) & 1;           // 奇数判断
311     int sx = x >> 31;                // x的符号(0为正, -1为负)
312     int sy = y >> 31;                // y的符号
313     int d = x + ~y + 1;               // x - y
314     int sd = d >> 31;                 // x-y的符号
315     int same = !(sx ^ sy);           // 是否同号
316     int gt = (same & !sd) | (!same & !sx);
317     return t + (odd & gt);
318 }
```

用 $t = (x \& y) + ((x \wedge y) \gg 1)$ ，计算向下取整的中点，再判断是否需要+1，+1 的条件为： $x+y$ 为奇数，且 $x>y$ ，同号看 $x-y$ 的符号，异号看 x 的符号。

12. P12

```
330 int isBetweenEitherOrder(int x, int a, int b) {
331     int sx = x >> 31, sa = a >> 31, sb = b >> 31;    // 符号位(0为正, -1为负)
332     int sameA = !(sx ^ sa), sameB = !(sx ^ sb);        // 判断是否同号
333     // 判断x>=a, 同号看x-a符号, 异号看x符号
334     int da = x + ~a + 1;
335     int geA = (sameA & !(da >> 31)) | (!sameA & !sx);
336     // 同理判断x>=b
337     int db = x + ~b + 1;
338     int geB = (sameB & !(db >> 31)) | (!sameB & !sx);
339     // (x>=a)^(x>=b), 再补端点
340     return (geA ^ geB) | !da | !db;
341 }
```

判断($x \geq a$ 或 $x \geq b$), 再用 $x == a$ 、 $x == b$ 补上端点即可。 $x \geq a$ 的比较类似 P11

13. P13

```
352 int mul5Sat(int x) {
353     int a = x << 2;                                // 4x
354     int y = a + x;                                  // 5x
355     int sh = (x >> 29) ^ (x >> 31);                // 移位溢出: 非0表示4x超出
356     int shmask = (~sh + 1) >> 31;                  // sh为0时为0, 非0时为-1, 非0表示溢出
357     int admask = (y ^ x) >> 31;                    // 加法溢出
358     int ovf = shmask | admask;
359     int sx = x >> 31;                                // x的符号
360     int min = 1 << 31;
361     int max = ~min;
362     return (y & ~ovf) | ((ovf & ~sx) & max) | ((ovf & sx) & min);
363 }
```

$x * 5$ 拆成 $4x + x$, 再判断溢出, 溢出分为移位产生的溢出和加法产生的溢出, 左移时若 $|x| \geq 2^{29}$ 则 $4x$ 溢出, 用 $(x >> 29) ^ (x >> 31)$, 非 0 则表示溢出, 加法时, 由于符号位是最高位, 加法溢出时, 同号相加会变号, 用 x 和 y 的符号判断是否溢出。最后正溢出取 `INT_MAX`、负溢出取 `INT_MIN`, 否则返回原结果。

14. P14

```
374 int classifyAdd3(int x, int y, int z) {
375     int sx = x >> 31, sy = y >> 31, sz = z >> 31;
376     int t = x + y;                                // x+y
377     int st = t >> 31;
378     int ovf1 = ~(sx ^ sy) & (st ^ sx);            // 溢出判断
379     int pos1 = ovf1 & ~sx;
380     int neg1 = ovf1 & sx;
381     int d1 = ~pos1 + 1 + neg1;
382     int s = t + z;                                // t+z
383     int ss = s >> 31;
384     int ovf2 = ~(st ^ sz) & (ss ^ st);
385     int pos2 = ovf2 & ~st;
386     int neg2 = ovf2 & st;
387     int d2 = ~pos2 + 1 + neg2;
388     return d1 + d2;
389 }
```

将三数相加拆成 $x+y$ 与 $(x+y)+z$ 两步。每步用 P13 相同的方法, 同号相加结果变号表示溢出, 再判断正负溢出的方式, 重复两次相加, 再将溢出合并即可。

15. P15

```
404 unsigned floatScaleThreeHalves(unsigned uf) {
405     unsigned sign = uf & 0x80000000;           // 符号位
406     unsigned exp  = (uf >> 23) & 0xFF;         // 指数位
407     unsigned frac = uf & 0x7FFFFFFF;          // 尾数位
408     if (exp == 0xFF) return uf;
409     unsigned M = (exp == 0) ? frac : (frac + 0x800000);
410     unsigned M3 = M + (M << 1);                // 3M
411     unsigned r  = (M3 + ((M3 >> 1) & 1)) >> 1;  // 3M/2向偶数舍入
412     unsigned newexp = exp;
413     if (r >= 0x1000000) {
414         r = (r + ((r >> 1) & 1)) >> 1;
415         newexp = exp + 1;
416     }
417     unsigned newfrac;
418     if (r >= 0x800000) {
419         newfrac = r - 0x800000;
420         if (newexp == 0) newexp = 1;
421     } else {
422         newfrac = r;
423         newexp = 0;
424     }
425     if (newexp >= 0xFF) return sign | 0x7F800000;
426     return sign | (newexp << 23) | newfrac;
427 }
```

先提取符号、指数、尾数，按要求对 NaN 原样返回。将尾数乘 3/2：先算 $3M = M + (M \ll 1)$ ，再用 $(X + ((X \gg 1) \& 1)) \gg 1$ 向偶数舍入除以 2 判断结果尾数，如果 $\geq 2^{24}$ ，则再向偶数舍入右移并指数加一，若结果在 $[2^{23}, 2^{24})$ 则为规格化数，否则为非规格化数，最后再处理指数溢出为无穷并重组位表示。

16. P16

```
441 unsigned floatRoundEven(unsigned uf) {
442     unsigned sign = uf & 0x80000000;
443     unsigned exp  = (uf >> 23) & 0xFF;
444     unsigned frac = uf & 0x7FFFFFFF;
445     if (exp >= 0xFF) return uf;           // 无穷
446     if (exp >= 150) return uf;           // 整数
447     if (exp < 127) {
448         if (exp == 126 && frac != 0)
449             return sign | 0x3F800000;
450         return sign;
451     }
452     int t = 150 - exp;
453     unsigned M = frac + 0x800000;
454     unsigned bias = (1 << (t - 1)) - 1;    // 2^(t-1) - 1
455     unsigned par  = (M >> t) & 1;
456     unsigned Mr   = ((M + bias + par) >> t) << t; // 舍入到 2^t 倍数
457     unsigned newexp = exp;
458     if (Mr >= 0x1000000) {
459         Mr >>= 1;
460         newexp = exp + 1;
461     }
462     return sign | (newexp << 23) | (Mr - 0x800000);
463 }
```

将浮点数舍入到最近整数，等价于把尾数 M 舍入到最近的 $2^{(150-\text{exp})}$ 的倍数，

类似 P10 的做法,用 $\text{bias}=2^{(\text{shift}-1)}-1$ 和奇偶性判断约束 $\text{par}=(M>>\text{shift})\&1$ 计算 $((M+\text{bias}+\text{par})>>\text{shift})<<\text{shift}$, 舍入结果达到 2^{24} 则右移一位并指数加一, 同时对 $|f|<1$ 的情况单独处理, $|f|\leq 0.5$ 舍到 ± 0 、 $0.5<|f|<1$ 舍到 ± 1 , NaN、无穷、以及整数直接返回。

17. P17

```

475 unsigned float_i2f(int x) {
476     unsigned sign = x & 0x80000000;
477     unsigned absx = x;
478     if (x < 0) absx = ~x + 1;
479     if (x == 0) return 0;
480     unsigned p = 31; // 找最高位
481     while (!((absx >> p) & 1)) p--;
482     unsigned exp = p + 127;
483     unsigned frac;
484     if (p <= 23) { // 无需舍入直接左移
485         frac = (absx << (23 - p)) & 0x7FFFFFFF;
486     } else { // p>23, 向偶数舍入
487         unsigned t = p - 23;
488         unsigned bias = (1 << (t - 1)) - 1;
489         unsigned par = (absx >> t) & 1;
490         unsigned mant = (absx + bias + par) >> t;
491         if (mant >= 0x1000000) {
492             mant >>= 1;
493             exp++;
494         }
495         frac = mant - 0x800000;
496     }
497     return sign | (exp << 23) | frac;
498 }

```

先取 x 的符号和绝对值, 再取最高位 p , 若 $p\leq 23$ 则尾数直接左移 $23-p$ 位并掩去隐含前导 1, 若 $p>23$ 则需右移 $t=p-23$ 位, 采用类似前几题的向偶数舍入 $(\text{absx} + 2^{(t-1)}-1 + ((\text{absx}>>t)\&1))>>\text{shift}$, 如果舍入结果达到 2^{24} 则右移一位并指数加一, 最后指数取 $p+127$, 加上符号返回。

18. P18

```

510 int bitCount(int x) {
511     int m1 = 0x55 | (0x55 << 8); m1 = m1 | (m1 << 16);
512     int m2 = 0x33 | (0x33 << 8); m2 = m2 | (m2 << 16);
513     int m3 = 0x0F | (0x0F << 8); m3 = m3 | (m3 << 16);
514
515     x = (x & m1) + ((x >> 1) & m1);
516     x = (x & m2) + ((x >> 2) & m2);
517     x = (x + (x >> 4)) & m3;
518     x = x + (x >> 8);
519     x = x + (x >> 16);
520     return x & 0x3F;
521 }

```

先用 $0x55555555$ 掩码将 32 位分成 16 对相邻 2 位, 再用 $(x\&\text{mask})+((x>>1)\&\text{mask})$

求每对内的 1 的个数, 之后同理, 不断合并把 1 的个数压缩到 x 中, 最后 $x \& 0x3F$ 得到总数

19. P19

```
532 int bitReverse(int x)
533 {
534     int mask;
535     mask = (0xFF << 8) | 0xFF;
536     x = ((x >> 16) & mask) | (x << 16);
537     mask = mask ^ (mask << 8);
538     x = ((x >> 8) & mask) | ((x & mask) << 8);
539     mask = mask ^ (mask << 4);
540     x = ((x >> 4) & mask) | ((x & mask) << 4);
541     mask = mask ^ (mask << 2);
542     x = ((x >> 2) & mask) | ((x & mask) << 2);
543     mask = mask ^ (mask << 1);
544     x = ((x >> 1) & mask) | ((x & mask) << 1);
545     return x;
546 }
```

先交换 16 位半字, 再依次交换 8 位、4 位、2 位、1 位组, 每级用交替掩码, $((x \gg k) \& \text{mask}) | ((x \& \text{mask}) \ll k)$ 把相邻两组对调, 实现交换。

附图

1. 在终端中执行 ./check_ops.py bits.c 后的截图

```
● xiongsy@localhost:~/icslab/xiongsy_Lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 5/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 11/16 operators
roundEvenPow2: 10/24 operators
midpointTowardFirst: 22/32 operators
isBetweenEitherOrder: 32/48 operators
mul5Sat: 23/30 operators
classifyAdd3: 28/52 operators
floatScaleThreeHalves: 19/60 operators
floatRoundEven: 18/65 operators
float_i2f: 29/40 operators
bitCount: 24/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
❖ xiongsy@localhost:~/icslab/xiongsy_Lab1$
```

2. 在终端中执行./btest 后的截图

```
● xiongsy@localhost:~/icslab/xiongsy_Lab1$ ./btest
Score  Rating  Errors  Function
1      1        0      signMask
2      2        0      bitXor
3      3        0      negativePart
4      4        0      copyByteWithin
4      4        0      logicalShift
4      4        0      swapNibblePairs
4      4        0      secondLowestZeroBit
5      5        0      oddParity
5      5        0      rotateRightBits
5      5        0      roundEvenPow2
5      5        0      midpointTowardFirst
7      7        0      isBetweenEitherOrder
7      7        0      mul5Sat
7      7        0      classifyAdd3
7      7        0      floatScaleThreeHalves
10     10        0      floatRoundEven
10     10        0      float_i2f
10     10        0      bitCount
10     10        0      bitReverse
Total points: 110/110
❖ xiongsy@localhost:~/icslab/xiongsy_Lab1$ █
```